

ECSE 343

Numerical Methods in Eng

Winter 2024

University of McGill

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Tianming Sun (Student #: 260894875)

Sijing Gu (Student #: 261044524)

Instructor: Prof. Roni Khazaka

Date: April 15th, 2024

1. Abstract

In this study, we explore and assess various numerical integration techniques to determine their efficacy and accuracy in addressing complex multidimensional challenges. Initially, we selected the quadrature method with the trapezoidal rule for its straightforward implementation and reasonable precision. However, during our experimental evaluations, the trapezoidal rule demonstrated limitations in handling two and three-dimensional integrations effectively. Consequently, we introduced the sparse grid method to our analysis. This method has shown promise in managing high-dimensional data efficiently while reducing computational demands. Through extensive testing, these methods were rigorously evaluated to confirm their appropriateness and performance in meeting our project's specific needs, culminating in their integration into our research framework.

2. Introduction

A vital tool in mathematics, engineering, and the applied sciences, numerical integration approximates integrals that are usually intractable using analytical techniques, particularly in multidimensional scenarios. The evaluation of integrals related to the modeling of complex physical occurrences, which frequently requires an abundance of resources and time, considerably escalates the level of difficulty of this computing task. The creation of precise and effective numerical integration methods that minimize the quantity of function evaluations is therefore crucial.

The Monte Carlo approach stands out among the other strategies for numerical integration because it utilizes random sampling to estimate integrals, which is particularly useful in high-dimensional settings. Even though there are many advantages to this approach, research continues to be conducted on discovering alternatives that might provide comparable accuracy at a lower computing cost.

This study aims to find the optimal balance between computational efficiency and precision by evaluating multiple methods for multidimensional numerical integration. We highlight two primary approaches: the Monte Carlo method and another selected for its efficiency in minimizing function evaluations. Utilizing a range of integrand functions implemented in MATLAB, this paper investigates the practicality and effectiveness of these methods within real-world engineering scenarios.

By tackling the computational challenges of multidimensional integration, this research advances the theoretical knowledge of numerical methods and offers practical insights that could improve computational strategies across engineering and related disciplines. Our goal is to leverage these insights to direct future research and practical applications of numerical integration, demonstrating the potential for innovative solutions that address the limitations inherent in traditional approaches.

3. Descriptions of the Methods

1. Monte Carlo Method

The Monte Carlo method of integration is based on the Law of Large Numbers. Taking random samples from the integrand's domain causes the average of these samples to converge to the expected value of the function, which is equivalent to the integral over the domain. The method is especially useful for high-dimensional integrals because, unlike traditional numerical integration methods, its convergence rate is not heavily influenced by the dimensionality of the function.

2. Quadrature Method with Trapezoidal Rule

Quadrature methods, such as Simpson's or Trapezoidal rule, approximate the integral by dividing the domain into small segments and adding the weighted function values at specific points within each segment. These methods rely on polynomial approximations for the function within each segment. The accuracy is determined by how well the function can be approximated by polynomials over each segment, and thus is heavily influenced by the function's smoothness.

3. Sparse grid method

The sparse grid method is similar to the trapezoidal quadrature. However, the quadrature methods calculate the function value on every point and sum them up in the end taking account of their weight. This causes a significant increase in computation effort and time when dimension goes up. Sparse grid alleviates this problem by using smolyak algorithm. This algorithm selects a set of points that matters the most and their weights are distributed based on their contribution. It forms a grid which is sparse and saves computational cost.

3.1. Pseudo-code of the three Methods

Sparse Grid Pseudocode	Trapezoidal Quadrature Pseudocode	Monte Carlo Method Pseudocode
Function smolyak_trap_3d(func, a, b, level) Initialize total integral and evaluation count Generate equidistant points along x For each x-point Initialize 2D slice integral and evaluations Generate equidistant points along y For each y-point Define 1D function along z Integrate 1D function using trapezoidal rule Update 2D slice integral and evaluations with weights at boundaries Update total integral and evaluations with weights at boundaries Scale total integral by step sizes Return total integral and total evaluations Function trapezoidal_rule(func, lowerBound, upperBound, n) Initialize sum with end points Sum values at intermediate points with appropriate weights Return scaled integral and evaluation count	Function multidimensionalTrapezoidal(f, d, n, low, high) Compute step size h and generate points pts Initialize edgeWeights to 0.5 and interiorWeight to 1 Initialize sum to 0 For each index in 1 to n^d: Convert linear index to multidimensional index ind Set coordinates x based on pts and adjust weight for edges Add weighted function value at x to sum Adjust sum for step size in all dimensions to get integral value End Function	Function MonteCarloIntegration(func, a, b, num_samples) Input: func (function to integrate), a (lower limits), b (upper limits), num_samples If length(a) != length(b) Then Error "The dimension of lower and upper limits must be the same" EndIf volume = Product(b - a) For each dimension i from 1 to length(a) random_points[:, i] = a[i] + (b[i] - a[i]) * Random(num_samples) EndFor For i from 1 to num_samples function_values[i] = func(random_points[i, :]) EndFor integral_estimate = volume * Mean(function_values) Return integral_estimate, num_samples EndFunction

Figure 1. Pseudocode for Three Method

3.2. Justification for choice of second non-Monte-Carlo method

Two types of integration methods exist in the literature other than Monte Carlo based methods, which are quadrature methods and sparse grid methods as in the description. We have implemented a trapezoidal quadrature method. We noticed that the number of function calls grows exponentially in 2-D and 3-D. This causes a large increase of computational cost. Research is conducted to measure the error bounds of different quadrature methods and it turns out that most of them hold the similar relative error. In order to reduce CPU cost, we decided to use sparse grid methods and use trapezoidal quadrature as a 1-D base. As the description mentioned, sparse grid methods should work much faster than quadrature since it only runs the function on a selected set of points.

3.3. Comparison of Three Methods

Comparison of Three Methods

Table 1. Advantages and Disadvantages of both Methods

Aspects/Methods	Monte-Carlo Method	Designed Quadrature Method with Trapezoidal Rule	Sparse Grid Method
Advantages	- Resilient to changes in dimensionality (free from the curse of dimensionality) - Simple to implement and parallelize. - Does not require that the function be smooth	- Simplicity for smooth functions - Efficiency for smooth functions - Deterministic approach - Error estimation - Compatibility with regular grids	- Simplicity for Smooth Functions - Efficiency for Smooth Functions - Deterministic Approach - Error Estimation. - Compatibility with Regular Grids
Disadvantages	- Slow convergence - Requires large number of samples to achieve high accuracy. - Results are inherently random that creates a variance that must be managed.	- Low accuracy for oscillatory or discontinuous functions - Sensitivity to function behavior - Higher computational cost for high-dimensional problems - Requirement for uniform sampling	- Low Accuracy for Oscillatory or Discontinuous Functions - Sensitivity to Function Behavior - Higher Computational Cost for High-Dimensional Problems - Requirement for Uniform Sampling

4. Evaluation of the Methods

Table 2. Integral Results of three Methods Compared to Built-in Method in MATLAB

Number of Func Eval = 100,000 for all three methods								
		Monte Carlo Method		Quadrature Method with trapezoidal rule		Sparse Grid Method		Builtin integral
		Estimated integral	Relative error	Estimated integral	Relative error	Estimated integral	Relative error	
Function 1	1D	0.9979	0.0600%	0.9973	0.0000%	0.997	0.0000%	0.9973
	2D	0.9892	0.5400%	0.9945	0.0133%	0.995	0.0003%	0.9946
	3D	0.9963	0.4500%	0.9895	0.2500%	0.992	0.0175%	0.9919
Function 2	1D	1.0000	0.0000%	1.0000	0.0010%	1.000	0.0010%	1.0000
	2D	1.0001	0.0100%	0.9906	0.9400%	1.000	0.0000%	1.0000
	3D	0.9965	0.3500%	0.9010	9.9000%	1.002	0.2200%	1.0000
Function 3	1D	11.8838	0.1600%	11.9029	0.0001%	11.903	0.0001%	11.9029
	2D	141.8079	0.0900%	141.4823	0.1400%	141.678	0.0007%	141.6790
	3D	1691.5190	0.3000%	1657.0000	1.7400%	1685.800	0.0365%	1686.3909
Function 4	1D	0.1916	0.0900%	0.1918	0.0007%	0.192	0.0007%	0.1918
	2D	-0.0176	8.2600%	-0.0159	2.4900%	-0.016	0.0083%	-0.0163
	3D	-0.0077	62.6000%	-0.0077	61.1300%	-0.005	0.4600%	-0.0048
Function 5	1D	0.4990	0.1900%	0.5000	0.0007%	0.500	0.0007%	0.5000
	2D	0.2504	0.1500%	0.2484	0.6300%	0.250	0.0000%	0.2500
	3D	0.1250	0.0080%	0.1164	6.8600%	0.125	0.0731%	0.1250
Function 6	1D	0.1669	0.1200%	0.1667	0.0001%	0.167	0.0001%	0.1667
	2D	0.0153	0.7100%	0.0151	0.0503%	0.015	0.0217%	0.0152
	3D	0.0010	1.6640%	0.0009	0.2500%	0.001	1.2900%	0.0009

Relative Error vs. Number of Function Evaluations for 3D @func 6 in MATLAB

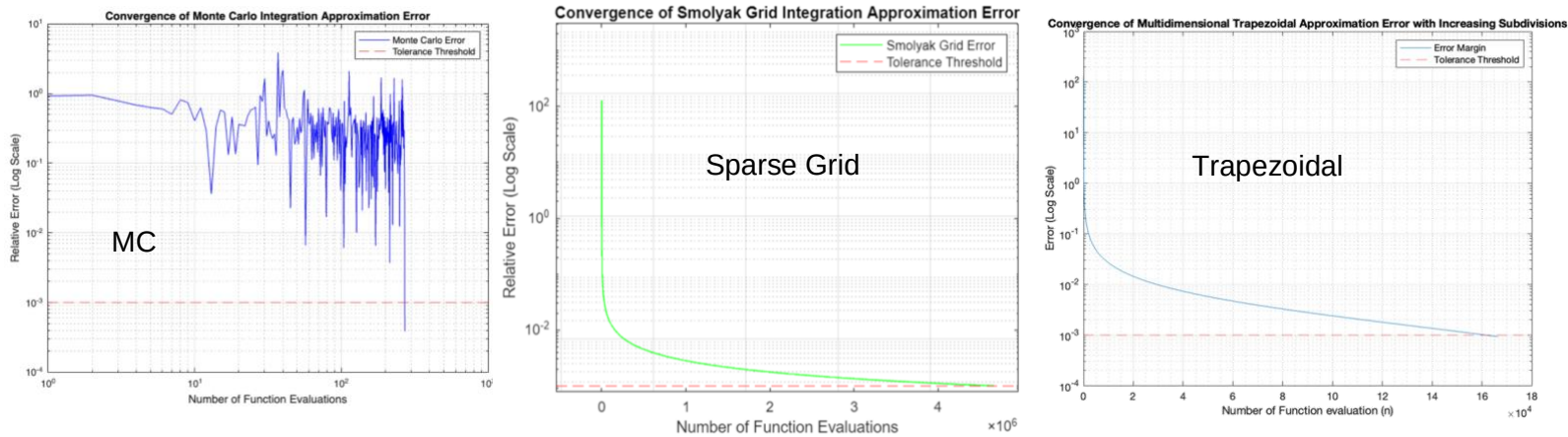


Figure 2. MATLAB Monte-Carlo Method (left), Sparse Grid Method (middle), Trapezoidal Method (Right) for function 6, 3D

5. Graph Analysis and Discussion

To compare the accuracy of each approach, the number of function calls is fixed to 100,000 calls, and the **relative errors to the build-in method result are used to determine the accuracy**. The results are tabulated in Table 2. The cells highlighted in **yellow** show where the accuracy of the applied method is low for a given case. The cells highlighted in **green** show the applied method is accurate at the given function.

The processing time varies depending on the CPU of each computer to run the codes. Therefore, **the number of function evaluations** is used as a **benchmark for the efficiency** of the program. To compare the efficiency of each approach, **the number of function evaluations to reach a relative error of 0.1% is used to evaluate the efficiency**. Figure 2 shows an example for function 6, 3D.

The Monte Carlo approach was assessed and shown to have substantial fluctuation in accuracy, notably for function 4 in 2D and 3D, and for function 6 in 3D, as highlighted as yellow in Table 2. These differences are mostly due to stochastic character and difficulties with complicated, higher-dimensional functions, which prevent effective convergence within a finite number of evaluations, since Monte Carlo is working overall well with other functions with relative errors roughly near 0.1%. Although this approach efficiently combats the curse of dimensionality, its low convergence rate can be inefficient. As demonstrated in Figure 2, the Monte Carlo has a very big variance as number of function iterated increases since it is a method that is very dependent on the random sample points. However, while Monte Carlo can attain higher accuracy by evaluating more functions, it incurs larger CPU costs.

For the trapezoidal quadrature, its accuracy for 100,000 function evaluations varies in different dimensions. As Table 2 shows, its accuracy for the first dimension is much better than Monte Carlo methods. This is the reason that it is chosen as the base case used in sparse grid method. This method shows different accuracies for different functions in 2D and 3D. It can be observed that it works best among the methods for function 6 but works worse for function 3, 2 and 5. In Figure 2, it reaches the 0.1% relative error at just 180,000 function evaluations. It is because this method tends to work better on function 6.

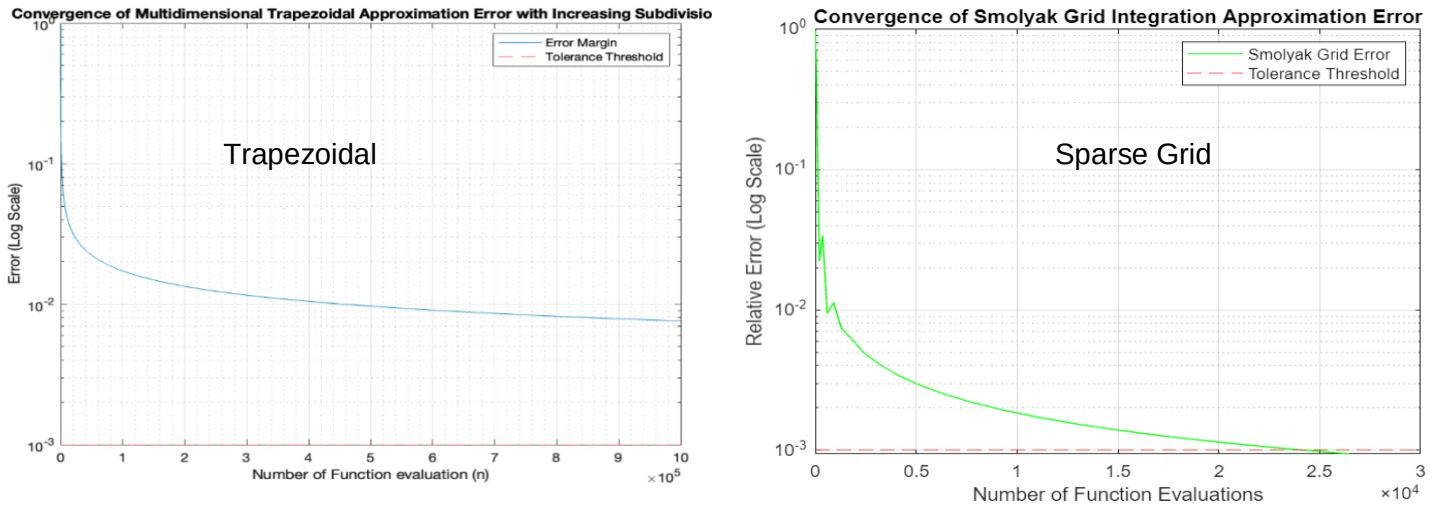


Figure 3. MATLAB Relative Error vs. Number of Function Evaluations for Function 3, 3D

In Sparse Grid method using Smolyak grid integration, increasing number of function evaluations increases the accuracy of the solution. Table 2 shows that the sparse grid method gives very high accuracy at 100,000 number of evaluations. The relative errors using sparse grid method, for function 1-6, 1D to 3D cases are almost always the smallest of all three methods, except for function 6, 3D and function 5, 3D. As shown in Figure 2 for function 6, 3D, at 4.6 million number of evaluations, the integral results reached a relative error of 0.1% tolerance level. For other cases, Sparse Grid method is very accurate and efficient,

except for function 6. For example, as shown in **Error! Reference source not found.**, for the case of function 3, 3D, the tolerance of $1e-3$ is reached using only 27,000 function calls using Sparse Grid method, while trapezoidal method takes more than 1,000,000 function calls. This, and other results from func1-5 shows that sparse grid far superior to the other two methods.

To conclude, the sparse grid method has the best overall accuracy in a given number of function evaluations. The trapezoidal method works excellent in 1D and holds an unstable accuracy that varies with functions in 2D and 3D. The Monte Carlo method, on the other hand, has the similar accuracies with the trapezoidal method. However, due to the random numbers involved in the algorithm, the accuracy must have some fluctuation from the mean value. This may sometimes cause some extreme values that differs a lot from the true value.

6. Conclusion

In our analysis of numerical integration methods, the sparse grid approach resulted as the best option, particularly under the limitations of reducing CPU usage and function evaluations while increasing accuracy. Employing the Smolyak algorithm, this technique effectively identifies critical points for examination, considerably lowering computing overhead while maintaining precision. This capacity makes it ideal for complicated, high-dimensional integrals, which precisely aligns with the objectives of obtaining high accuracy in multidimensional numerical integration with little processing resources.

7. Contribution

Table 3. Contributions of each Member

Member	Role
Sijing Gu (261044524)	- Implementation of the 3 functions in MATLAB - Implementation of designed quadrature using trapezoidal rule - Description of sparse grid method - Testing the performance of trapezoidal method - Make seats for friends
Tianming Sun (260894875)	- Implementation of the other 3 functions in MATLAB - Implementation of designed quadrature method using trapezoidal rule - Implementation of sparse grid method - Efficiency and accuracy comparison implementations - Write up of report and formatting
Nara Yun (261115268)	- Front page - Implement a Monte-Carlo based approach - Description of the methods, Advantages and disadvantages of Monte-Carlo Algorithm - Testing the performance of Monte-Carlo method

	- Advantages and Disadvantages of Designed Sparse Grid Method
Karen Chen Lai (261107674)	- Abstract - Introduction - Formatting report - Advantages and Disadvantages of Designed Quadrature Method with Trapezoidal Rule - Implement a Monte-Carlo based approach - Discussion for Monte-Carlo Integration

8. References

1. M. Mascagni and A. Srinivasan, "Algorithm 806: SPRNG: A Scalable Library for Pseudorandom Number Generation," in ACM Transactions on Mathematical Software, vol. 26, no. 3, pp. 436-461, 2000. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574002196010179>
2. P. Hellekalek, "Don't Trust Parallel Monte Carlo!," in 12th International Conference on Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing, Springer, Dordrecht, 1998, pp. 87-112. [Online]. Available: https://link.springer.com/chapter/10.1007/978-94-011-5014-9_7
3. G. S. Fishman, "Monte Carlo: Concepts, Algorithms, and Applications," SIAM Review, vol. 6, no. 1, pp. 1-63, 1964. [Online]. Available: <https://epubs.siam.org/doi/epdfplus/10.1137/0106028>
4. M. Griebel, "Sparse Grids and Related Approximation Schemes for Higher Dimensional Problems," in Foundations of Computational Mathematics, Hong Kong 2000, Springer, Berlin, Heidelberg, 2001, pp. 106-161. [Online]. Available: <https://link.springer.com/article/10.1023/a:1019129717644>
5. Maxwell Rules, "Monte Carlo Integration." [Online]. Available: https://maxwellrules.com/math/montecarlo_integration.html
6. User "copper.hat," Answer to "Monte Carlo Pseudocode by Richard S. Sutton and Andrew G. Barto," Stack Overflow, 2022. [Online]. Available: <https://stackoverflow.com/questions/72946080/monte-carlo-pseudocode-by-richard-s-sutton-and-andrew-g-barto>
7. C. Zechner and M. Granitzer, "Efficient Integration and User Guidance for Monte Carlo Tree Search," Statistics and Computing, vol. 26, no. 1-2, pp. 103-113, 2016. [Online]. Available: <https://link.springer.com/article/10.1007/s11222-015-9596-z>
8. MathWorks, "Monte Carlo Estimation Examples with MATLAB." [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/55306-monte-carlo-estimation-examples-with-matlab>
9. Encyclopedia of Mathematics, "Smolyak Algorithm." [Online]. Available: https://encyclopediaofmath.org/wiki/Smolyak_algorithm
10. L. Plaskota and W. Römisch, "Noisy Information and Computational Complexity," Mathematics, vol. 11, no. 19, 4191, 2023. [Online]. Available: <https://www.mdpi.com/2227-7390/11/19/4191#FD1-mathematics-11-04191>

11. Maruri-Aguilar, H., Wynn, H. Sparse polynomial prediction. *Stat Papers* **64**, 1233–1249 (2023).
<https://doi.org/10.1007/s00362-023-01439-8>